Aviation Safety Atlas

System Architecture & Implementation Blueprint

Lead Engineer: Maik Geijsen | **Milestone Target:** Mid-July 2026 | **Stack:** Vite / React / FastAPI

Current artifact note: the apps/web scaffold exists in atlas-web-app-docs-reports.zip and the BFF skeleton exists in atlas-bff-docs-reports.zip. Phase 1 is wire and harden, not build from scratch.

1. Executive Summary

The **Aviation Safety Atlas** is transitioning from a backend prototype to a production-ready defensible-record system. This blueprint supersedes earlier iterative plans by building directly upon the recently validated React/Vite frontend scaffold. The immediate objective is to harden this frontend, secure the API surface through a Backend-for-Frontend (BFF), and construct the critical modules for unstructured document ingestion and court-submissible report generation. The end result is a system where every single projected fact is strictly accountable to an immutable evidence chain.

2. Core Architectural Foundations

> *"The projected public record is not the truth. The evidence chain is the truth. The projection is the current best explanation of that evidence."*

The system leverages a clean-architecture stack designed for aggressive data integrity, auditable decision workflows, and strict multi-tenant isolation:

- **Backend Data Integrity:** Powered by FastAPI, SQLAlchemy async, and PostgreSQL/PostGIS. The underlying data model is strictly append-only, ensuring total auditability of claim histories, supersession chains, and conflict resolutions.
- **The Frontend Scaffold:** A lightweight, fast Vite, React, TypeScript, and Tailwind CSS application utilizing TanStack Query to manage complex server-state interactions like chronological timelines and high-density evidence data arrays.

The BFF Security Boundary

The browser-based Single Page Application (SPA) must never directly handle raw `X-API-Key` payloads. A secure, Node-based Backend-for-Frontend (BFF) sits between the user and the FastAPI core. Users authenticate at the BFF, generating an `HttpOnly`, `SameSite=Lax` session cookie. The BFF maps this session to the user's server-side API key and proxies requests to FastAPI, seamlessly interacting with the existing Row-Level Security (RLS) and Role-Based Access Control (RBAC) mechanisms.

3. Key Workflows & UI Surfaces

The Conflict Queue (The "Killer Screen")

When multiple investigation sources (e.g., NTSB vs. FAA records) disagree on a field, the system surfaces an explicit OPEN conflict rather than silently resolving the discrepancy. Reviewers must select a winning claim or input a manual override on the record with a required rationale. Crucially, the UI leverages optimistic concurrency: if a backend `HTTP 409 (ConflictModifiedError)` is thrown, the UI catches it, displays a refreshed state inline, and forces the reviewer to re-evaluate based on the most current data.

Unified Evidence Panel

A unified, persistent component utilized across claims, conflicts, and the Chronos timeline. This panel displays the original source, raw extracted values, reliability tier, ingestion timestamp, and cryptographic audit hash. This guarantees that the entire provenance chain is persistently accessible without breaking the user's workflow.

4. Resolving Critical Functional Gaps

To reach a sellable, complete MVP, two highly specialized backend layers must be developed in the short term:

Module	Implementation Strategy & Purpose
Unstructured Document Ingestion PDFs Dockets	Following the design pattern of the NTSB eADMS importer, this module will process PDF dockets and hearing transcripts. It employs a conservative extraction strategy using `pypdf` for plain text, preserving the full raw byte payload in S3-compatible object storage to guarantee legal defensibility against extraction hallucination.
Report Export Engine WeasyPrint HTML/PDF	A read-only rendering engine that reads the projection, provenance, and conflict history to generate immutable, version-controlled PDF reports via `WeasyPrint`. Every single output fact includes an explicit footnote tracing back to its ingestion timestamp and reliability tier.

5. Recommended Development Phasing

Execution should prioritize securing the base and proving the core loop before introducing complex document parsing.

1. **Phase 1: Foundation & Security Boundary.** Establish the BFF session architecture and verify an end-to-end, authenticated data passthrough.
2. **Phase 2: The Working Surface.** Construct the three-column workspace layout, wire up the conflict queue mechanics, and implement strict HTTP 409 error handling.
3. **Phase 3: Evidence Integration.** Build the S3-backed document ingestion pipelines and the immutable report export modules.
4. **Phase 4: Hardening & Launch.** Finalize security audits, complete enterprise compliance documentation, and ship the public-facing lead capture site.